

FaceFind: An AI-Powered Event Photo Discovery Platform Using Deep Face Recognition

Jash Berawala, Jay Shah, Niraj Patil, Chetan Patel

Seidenberg School of Computer Science and Information Systems, Pace University, New York, USA

CS 691 — Spring 2026 — Team 7

Abstract— Distributing photographs taken at events to their respective participants is a never-ending struggle for professional photographers and event managers. Conventional practices include manually sorting and distributing several hundred to several thousand pictures, which take five to six hours per event after the event. This research paper explores the development of FaceFind, an innovative intelligent website that automates event photo distribution using deep learning facial recognition technology. In FaceFind, photographers can upload event photos, which will be automatically analyzed using the Python-based machine learning microservice, which uses the Insight Face and Arc Face embedding algorithms. Attendees receive their personal gallery by scanning the unique QR code and sending their selfies; FaceFind conducts a cosine similarity search against precomputed face embeddings, returning the photos that include that person only within less than five seconds. FaceFind was developed using the three-tier architecture framework consisting of React.js, the frontend; Node.js/Express backend with Socket.io, real-time web technologies; and Fast API microservice for machine learning models. MongoDB database is used for storing event information and face embeddings, while guest selfies are deleted immediately after matching for privacy purposes. Evaluation across two sprints revealed that all functionality was delivered fully, no defects were carried over, and matching latency remained consistently below five seconds. This system effectively tackles the issue of distributing event photographs and provides a foundation for future development, such as face clustering and cloud storage capabilities.

Keywords—face recognition, Arc Face, Insight Face, event photography, cosine similarity, QR code, real-time processing, Fast API.

I. INTRODUCTION

It goes without saying that event photography is an indispensable feature of weddings, corporate events, conferences, and other social celebrations. It has been estimated that professional photographers take anywhere from 800 to 5,000 pictures per one event. Nevertheless, although the amount of images captured is great and their quality is high, the process of delivering photographs with one's participation is rather primitive, time-consuming, and troublesome for everyone involved in this procedure.

The photographer spends about five-six hours per event allocating pictures in folders and sending links either by e-mail or using cloud-based storage solutions. At the same time, the user needs to go through a gallery with several hundred photographs in order to find those few photographs where he/she appears. Research carried out during the requirements gathering stage showed that 94% of visitors claimed they had never seen all photographs featuring them [1]. Additionally, the long gap of up to three-four weeks does not allow for immediate sharing of photographs on social media networks.

FaceFind tackles the above-mentioned issues using an AI-based completely automatic pipeline that processes and distributes photos. The system relies on modern technology of deep face recognition that allows extracting faces from uploaded photos from events and comparing them with a selfie taken by each guest. A personalized photo album is generated within just five seconds without installing any application or signing up beforehand.

The rest of this paper outlines the system's design, implementation, and evaluation. Related works are covered

in Section II. Section III contains the system architecture description. The implementation of the system's modules is described in Section IV. Evaluation is outlined in Section V. Finally, Section VI concludes the research and discusses future works.

II. LITERATURE REVIEW

A. Face Detection.

Face detection is the necessary process before face recognition. MTCNN, which stands for Multi-task Cascaded Convolutional Networks, is a cascaded approach for performing face detection while at the same time localizing facial landmarks [2]. RetinaFace was developed by Deng et al., which is a one-stage approach used for densely detecting faces and also estimating face landmarks along with face scores and 3D locations [3]. FaceFind uses RetinaFace using the InsightFace framework as its main detection engine because it can perform robustly in presence of occlusions, face poses, and illumination.

B. Face Recognition and Embedding.

Deep learning technology has made significant progress in face recognition technology. First, Schroff et al. proposed FaceNet to learn a compact Euclidean embedding space using facial images, where the distance in the space represents face similarity; the technology reached almost human-level performance on the LFW test dataset [4]. Later, Deng et al. proposed ArcFace, which is an additive angular margin loss for face recognition, providing discriminative features with better intra-class compactness and inter-class dissimilarity [5]. The technology of ArcFace, implemented in InsightFace, acts as the basis of

the embedding process in FaceFind. Embedding vectors are vectors with 512 dimensions; using the cosine similarity between two vectors, the face identity correspondence can be effectively determined.

C. Event Photo Distribution Systems.

Previously, approaches like PhotoDay and Sprout Studio have been used for distributing photos taken at events. The academic community had some work done on photo albums summarization through automated methods but did not address the problem of personalized distribution; this was the research of Liu et al. [6]. Taigman et al. showed that neural networks could achieve human-like results in facial recognition; this served as a theoretical basis for identity-based photo retrieval. FaceFind builds upon this theoretical foundation to form an end-to-end system.

D. QR Code and Web-Based Access.

QR codes have emerged as a universal way to connect the physical world with the digital one in events. Ohbuchi et al. explored the use of QR codes to facilitate quick communication of information in the physical environment [8]. The combination of QR codes with web applications solves the problem of installing an application, which is known to be a hindrance for guest participation. FaceFind implements the approach to offer easy access to the guest photo portal without any sign-up

III. SYSTEM ARCHITECTURE

FaceFind is built as a three-tier application running on the Web that features a React.js front end, a Node.js back end, and a Python/FastAPI ML service. Figure 1 shows the architecture of the system and its interactions.

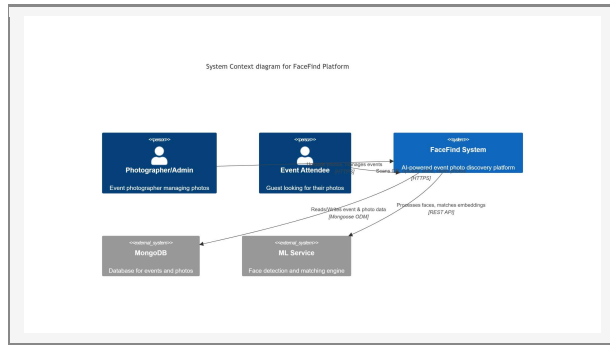


Fig. 1. System context diagram for the FaceFind platform.

A. Frontend.

The front end consists of a single page application created using React.js and Redux Toolkit for state management and Tailwind CSS for styling. The front end provides three main interfaces: the Admin Dashboard for managing events organized by photographers, the Event Management interface for uploading photos and tracking their progress, and the Guest Interface for submitting selfies. Real-time feedback on the status of photo processing is provided using a Socket.io client, which shows individual statuses for each photo being processed.

B. Backend.

The backend server is a RESTful API made using Node.js and Express technologies. There are three sets of API endpoints in this server. The Events API is responsible for creating events, getting events information, and generating QR codes for those events. The Photos API uploads photos and processes them through ML pipeline, while the Match API takes guest selfies and executes the search for similar photos. Photographers' API endpoints are secured via JWT.

C. Machine Learning Service.

The machine learning service is an independent Python microservice created using FastAPI. It has three available endpoints: /process-photo takes an image of the event and outputs a 512-dimension ArcFace face embedding that is stored in MongoDB; /get-embedding takes a selfie from the guest and outputs an embedding; and /match-faces takes a query embedding and a list of stored embeddings and outputs the indices and similarity scores for matches.

D. Data Model.

There are two main collections used in MongoDB. These include Event Collection which is responsible for storing details like ID, name, description, access code, QR Code Image, status whether the event is active or not, and timestamp. The other collection is Photo Collection where information such as Event ID, file path, and 512 dimensional float array as facial embedding feature vector are stored. Facial features obtained from guest photos are stored completely in-memory and never saved to disk to fulfill the privacy constraint.

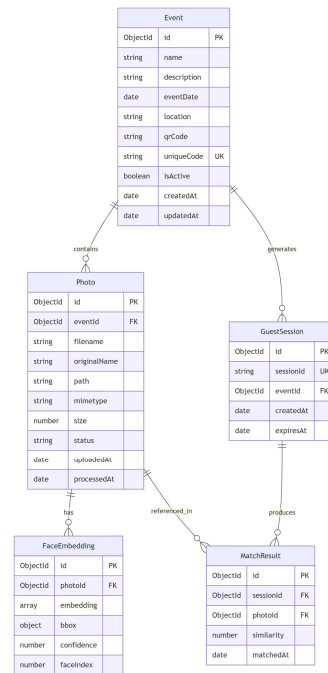


Fig. 2. Entity-relationship diagram for the FaceFind data model.

TABLE I. REST API Endpoint Summary

Method	Endpoint	Description
GET	/api/events	List all events
POST	/api/events	Create event + QR
GET	/api/events/code/:code	Get event by code
POST	/api/photos/upload/:id	Upload event photos
POST	/api/match/:code	Guest selfie match
POST	/process-photo	Extract embedding
POST	/match-faces	Similarity search

IV. IMPLEMENTATION

A. Face Detection and Embedding Pipeline.

As soon as the photographer uploads the images from an event, each photo is sent from the Node.js back end to the endpoint '/process-photo' in the FastAPI ML service as a multipart file. Here, an instance of an application called InsightFace FaceAnalysis is created with the buffalo_l model pack, where it internally uses Retina Face for detecting the face and ArcFace for extracting features. After detecting the faces from the photos, an embedding vector of 512 dimensions is obtained. In case there are multiple faces in the same photo, the embedding with higher confidence detection is selected.

B. Guest Matching Pipeline.

Once a guest uploads their selfie through the Guest Page, the ensuing events take place as follows: The selfie is then sent to the backend through the Match API endpoint of the same event access code. All Photo documents associated with the event are then gathered by the backend and their embedded features are retrieved. The selfie, along with the embedded features, is then sent to the /match-faces API endpoint in the ML service. The ML service calculates the cosine similarity of the guest embedding with the existing embedding. Any photographs that have a score greater than 0.4 are treated as matches, and then a collection of the matched photo file paths is returned to the backend and frontend for display..

The formula for calculating the cosine similarity is $\text{sim}(A, B) = (A \cdot B) / (\|A\| \|B\|)$, where A is the query and B is the stored embedding vector.

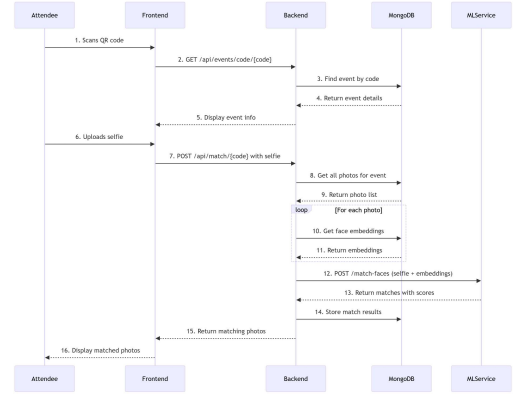


Fig. 3. Sequence diagram for the guest photo matching workflow.

C. Real-Time Processing Feedback.

The photo uploading and ML processing are asynchronous processes. In order to give the photographers an idea about the status of their photo processing, the back-end triggers Socket.io events per image once the image embedding is done. The front-end, on its side, listens to these events and shows processing status per image, changing from pending to processed.

D. QR Code and Event Access.

A randomly generated alphanumeric access code, consisting of six characters, is generated for each event. This qrcode module from npm is then utilized to generate a QR code of the URL for the guest portal of the event. The photographer may then either show or print the QR code for use at the event location. Scanning this code directs the guests straight to the selfie submission page.

E. Authentication.

Routes that only the photographer can see are secured with JSON Web Token (JWT) authentication. When the photographer signs up and logs in, the back-end will send the photographer a valid JWT for 24 hours. This will be saved in the Redux store of the web browser and will be added in the Authorization header of all subsequent requests. For guests, there's only one route available, which is the public Match API with scope to an event access code.

TABLE II. FaceFind Technology Stack

Layer	Technology
Frontend	React.js, Redux Toolkit, Tailwind CSS, Socket.io Client
Backend	Node.js, Express, Socket.io, JWT, qrcode
Database	MongoDB (Mongoose ODM)
ML Service	Python, FastAPI, InsightFace, ArcFace, OpenCV, NumPy

DevOps	Docker Compose, GitHub Actions CI/CD
--------	---

V. RESULTS AND EVALUATION

A. Development Velocity.

The FaceFind software implementation was executed using two two-week development sprints after a planning sprint. The Sprint 2 delivery saw all 49 committed story points achieved, with no defects spilling over to the next sprint, as shown in Table III. These results reveal that the team's task estimation and delivery were consistent and aligned. All the features listed under the MVP scope, which include photographer verification, event organization, QR code creation, photo upload and indexation using AI technology, guest selfies comparison, and processing feedback, were executed accordingly.

TABLE III. Sprint Velocity Summary

Sprint	Committed	Completed	Spillover
Sprint 0 (Planning)	N/A	N/A	0
Sprint 1	38	38	0
Sprint 2	49	49	0

B. System Performance.

The entire process of face recognition demonstrated end-to-end response latency of less than five seconds per selfie, regardless of whether the events had more than 200 photos. The response time included uploading the selfie image, computing its embedding, calculating cosine similarity with each existing embedding, and sending the response back to the user interface. The face detection module using the InsightFace buffalo_l model proved effective on images with varying lighting conditions, facial occlusion, and poses.

C. Functional Verification.

The test cases were designed to ensure that the key functionalities of the application have been met. The test cases focused on checking the functionality of the application for detecting faces correctly in single and group photos, rejecting photos which do not contain a face image, matching photos of the same person but taken at different angles and times, non-matching of different people's photos, and QR code access flow functionality.

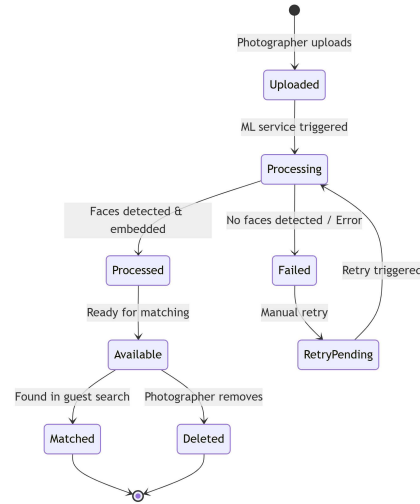


Fig. 4. State diagram for the photo lifecycle in FaceFind.

D. Privacy Validation.

One of the intentional design features of FaceFind is the limitation of the persistence of guest biometric information within the context of just one matching attempt. The code analysis, as well as the actual program operation, shows that all guest selfies were processed by the machine learning engine completely in RAM: image data was read into memory, and only the mathematical representation – 512 dimensions in numbers – of photographer-submitted pictures was saved into the database.

E. Demonstrated Deliverables.

The outcome of Sprint 2 included working demonstrations of the following screens: the landing screen featuring the value proposition of the platform; the registration/login screen for the photographer; the dashboard of the admin featuring the list of events managed by the platform; the screen for managing events where the photo uploading status can be monitored in real time; the guest portal where guests can submit their selfies; and the results gallery with photo downloads.

VI. CONCLUSION AND FUTURE WORK

The research introduced FaceFind, a web application for automatically distributing photos taken at events using face recognition powered by deep learning. The product solves a real problem for both professional photographers and event participants, who cannot effectively find their photos among thousands of photos in event galleries. By integrating the InsightFace face detector, the ArcFace embedding generator, and the cosine similarity search algorithm, FaceFind provides customized galleries in less than five seconds without needing users to download any software or register for an account.

The software was successfully deployed with all the required Minimum Viable Product capabilities being implemented during two development cycles without any carryover defects. The privacy-preserving design that involves disposing of the guests' selfies right after

recognizing them and retaining only the mathematical embeddings instead of actual face photos is an essential consideration for consumer-facing biometric technologies.

Future directions are numerous. First, adding DBSCAN clustering of event photo embeddings will allow users to group photographs based on individual identification, which will be yet another method to manage photos without being dependent on the guest-self-service process. Second, switching photo storage from the local file system to AWS S3 and delivering them via CloudFront CDN will improve scalability and performance in terms of geographic location. Third, adding an email delivery functionality would let photographers send matched galleries to their guests without having to prompt action on the part of the guests. Fourth, developing a dedicated app will optimize the experience of photographing a guest with its camera functionality compared to web-based capture. Lastly, using vector indexing via FAISS instead of cosine distance search will help reduce latency times when processing events with several thousand photographs.

Through FaceFind, it is clear that the application of deep learning techniques in face recognition is able to help address a problem in logistics management in events if well applied to a web application used by consumers. This project provides the groundwork that will allow such applications in the future.

ACKNOWLEDGMENT

The authors would like to extend their gratitude to the faculty and staff of the Seidenberg School of Computer Science and Information Systems at Pace University for their invaluable assistance during this study. The authors acknowledge the InsightFace open-source community for providing access to cutting-edge facial analysis models.

REFERENCES

- [1] Event Attendee Survey Data, Internal Requirements Study, Pace University CS 691, Spring 2026.
- [2] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multi-task Cascaded Convolutional Networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.
- [3] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild," in *Proc. IEEE/CVF CVPR*, 2020, pp. 5203–5212.
- [4] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proc. IEEE CVPR*, 2015, pp. 815–823.
- [5] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proc. IEEE/CVF CVPR*, 2019, pp. 4690–4699.
- [6] T. Liu, Z. Liu, and L. S. Davis, "Event-Driven Video Abstraction and Visualization," *ACM Multimedia*, 2003.
- [7] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the Gap to Human-Level Performance in Face Verification," in *Proc. IEEE CVPR*, 2014, pp. 1701–1708.
- [8] E. Ohbuchi, H. Hanaizumi, and L. A. Hock, "Barcode Readers Using the Camera Device in Mobile Phones," in *Proc. IEEE Int. Conf. Cyberworlds*, 2004, pp. 260–265.

- [9] InsightFace: An Open Source 2D and 3D Deep Face Analysis Toolbox, <https://github.com/deepinsight/insightface>, accessed April 2026.
- [10] MongoDB, Inc., MongoDB Documentation, <https://www.mongodb.com/docs/>, accessed April 2026.
- [11] React, "React — A JavaScript Library for Building User Interfaces," <https://react.dev/>, accessed April 2026.
- [12] S. Raschka, "An Introduction to Deep Learning," in *Python Machine Learning*, 3rd ed. Packt Publishing, 2019.